GitHub

The Complete Beginner-to-Confident Guide

Git basics, GitHub features, workflows, pull requests, security, cheat sheet and glossary

Contents

1. What is GitHub (and how is it different from Git)
2. Getting started: account, install, setup
3. Core concepts
4. Essential Git commands
5. Everyday workflow
6. Branching and merging
7. Forks and pull requests (contributing to open source)
8. Issues, Projects, Discussions, Wiki
9. Repository files: README, .gitignore, LICENSE
10. Major GitHub features
11. Collaboration, roles and code review
12. Undoing mistakes
13. Resolving merge conflicts
14. Best practices
15. Common problems and fixes
16. Quick cheat sheet
17. Glossary

1. What is GitHub?

Git is a free, open-source *version control system* created by Linus Torvalds in 2005. It runs on your computer and tracks every change you make to your files, so you can go back to any earlier version, work on features in parallel, and merge work from many people.

GitHub is a website/platform (owned by Microsoft since 2018) that *hosts Git repositories online* and adds collaboration tools on top: pull requests, issues, code review, automation, project boards, and more. Alternatives include GitLab and Bitbucket.

	Git	GitHub
What it is	Command-line tool / version control software	Cloud platform hosting Git repositories
Where it runs	Locally on your machine	In the browser and online
Needs internet?	No (except to push/pull)	Yes
Main purpose	Track changes in files	Share, collaborate, review, automate

Why use it?

- Full history of your project and the ability to undo mistakes
- Safe parallel work using branches
- Backup of your code in the cloud
- Teamwork: reviews, discussions, issue tracking
- Portfolio: recruiters look at your GitHub profile
- Home of most open-source software in the world

2. Getting Started

Step 1: Create an account

Sign up at github.com with email, username and password. Enable two-factor authentication (2FA) under Settings > Password and authentication.

Step 2: Install Git

- **Windows:** download Git for Windows from git-scm.com
- **macOS:** run `brew install git` or install Xcode command line tools
- **Linux (Debian/Ubuntu):** `sudo apt install git`

Step 3: Configure your identity

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git config --list
```

Step 4: Authenticate with GitHub

GitHub no longer accepts account passwords for Git operations over HTTPS. Use one of:

- **HTTPS + Personal Access Token (PAT) or Git Credential Manager** - easiest on Windows/macOS

- **SSH keys** - set up once, no repeated prompts
- **GitHub CLI** (`gh auth login`) - guided login

```
# SSH key setup
ssh-keygen -t ed25519 -C "you@example.com"
# add ~/.ssh/id_ed25519.pub to GitHub: Settings > SSH and GPG keys
ssh -T git@github.com     # test the connection
```

3. Core Concepts

Term	Meaning
Repository (repo)	A project folder tracked by Git, containing files and full history.
Commit	A saved snapshot of your changes with a message and unique ID (hash).
Branch	An independent line of development. Default branch is usually **main**.
Working directory	Your actual files on disk.
Staging area (index)	A holding zone where you choose which changes go into the next commit.
Remote	A version of the repo hosted elsewhere, usually named **origin**.
Clone	Download a full copy of a remote repo to your computer.
Fork	Your own copy of someone else's GitHub repo under your account.
Push / Pull / Fetch	Push uploads commits; fetch downloads without merging; pull = fetch + merge.
Pull Request (PR)	A request to merge your branch into another, with review and discussion.
Merge	Combining changes from one branch into another.
Issue	A tracked task, bug report or feature request.
HEAD	Pointer to the commit/branch you are currently on.
Tag / Release	A named marker for a specific commit, such as v1.0.

The three areas of Git

```
Working Directory --git add--> Staging Area --git commit--> Local Repo --git push--> GitHub (remote)
        <-----------------------------------git pull / git fetch--------------------------------
```

4. Essential Git Commands

Command	What it does
git init	Create a new repo in the current folder
git clone <url>	Copy a remote repo locally
git status	Show changed, staged and untracked files
git add <file> / git add .	Stage a file / stage everything
git commit -m "msg"	Save staged changes as a commit
git log --oneline --graph	Compact history view
git diff	Show unstaged changes (add --staged for staged)
git branch	List branches (add a name to create one)

Command	What it does
git switch <branch>	Move to a branch (older: git checkout)
git switch -c <new>	Create and switch to a new branch
git merge <branch>	Merge a branch into the current one
git remote -v	List remotes
git remote add origin <url>	Connect local repo to a remote
git push -u origin <branch>	Upload branch and set upstream
git pull	Fetch and merge remote changes
git fetch	Download remote changes without merging
git stash / git stash pop	Temporarily shelve / restore uncommitted work
git restore <file>	Discard changes in a file
git tag v1.0	Create a tag

5. Everyday Workflow

A) Start a brand new project and put it on GitHub

```
mkdir my-project && cd my-project
git init
echo "# My Project" > README.md
git add .
git commit -m "Initial commit"
# create an empty repo on github.com first, then:
git remote add origin https://github.com/USERNAME/my-project.git
git branch -M main
git push -u origin main
```

B) Work on an existing project

```
git clone https://github.com/USERNAME/repo.git
cd repo
git switch -c feature/login-page
# ...edit files...
git add .
git commit -m "Add login page"
git push -u origin feature/login-page
# open a Pull Request on GitHub
```

C) Stay up to date

```
git switch main
git pull origin main
```

6. Branching and Merging

Branches let you experiment without touching stable code. Create one per feature or bug fix, then merge it back when finished.

```
git switch -c feature/navbar     # create + switch
git add . && git commit -m "Build navbar"
git switch main
git merge feature/navbar         # bring changes into main
```

```
git branch -d feature/navbar     # delete local branch
git push origin --delete feature/navbar   # delete remote branch
```

Merge strategies on GitHub

Option	Result
Create a merge commit	Keeps all commits and adds a merge commit. Full history preserved.
Squash and merge	Combines all PR commits into one clean commit on main.
Rebase and merge	Replays commits on top of main for a linear history.

Rebase rewrites history to place your commits on top of another branch. Never rebase commits you have already shared with others on a public branch.

7. Forks and Pull Requests

Use this flow when you do not have write access to a repository (typical for open source, student projects and events like Hacktoberfest).

Step by step

- **Fork:** click Fork on the original repo to copy it into your account.
- **Clone your fork:** `git clone https://github.com/YOU/repo.git`
- **Add upstream** so you can sync: `git remote add upstream https://github.com/OWNER/repo.git`
- **Create a branch** for your change.
- **Commit and push** the branch to your fork (origin).
- **Open a Pull Request** from your branch to the original repo's main branch. Click 'Compare & pull request' on GitHub.
- **Respond to review:** push more commits to the same branch; the PR updates automatically.
- **Merge:** a maintainer merges the PR. Then sync your fork and delete the branch.

```
# sync your fork with the original repo
git fetch upstream
git switch main
git merge upstream/main
git push origin main
```

A good pull request

- Small and focused on one change
- Clear title and description: what changed and why
- Links the issue it resolves (write `Closes #12`)
- Passes tests and follows the project's contributing guide (CONTRIBUTING.md)
- Includes screenshots for UI changes

8. Issues, Projects, Discussions, Wiki

Feature	Use it for
Issues	Bugs, tasks, feature requests. Support labels, assignees, milestones and templates. Close automatically with 'Fixes #N' in a PR.
Projects	Kanban boards and tables to plan and track work across issues and PRs.
Discussions	Community Q&A, ideas and announcements that are not tasks.
Wiki	Longer documentation pages for a repo.
Milestones	Group issues toward a target such as a release.
Labels	Categorise issues: bug, enhancement, good first issue, help wanted.

9. Repository Files

README.md

The front page of your repo, written in Markdown. Include: project name, description, features, installation, usage, screenshots, contributing and license.

```
# Project Title
Short description.

## Features
- Feature one
- Feature two

## Installation
```bash
git clone https://github.com/user/repo.git
npm install
```

License
MIT
```

.gitignore

Lists files Git should not track: dependencies, build output, secrets, OS junk.

```
node_modules/
.env
*.log
__pycache__/
.DS_Store
dist/
```

LICENSE

Tells others what they may do with your code. Common choices: **MIT** (very permissive), **Apache 2.0** (permissive with patent grant), **GPL-3.0** (copyleft; derivatives must stay open). Without a license, default copyright applies and others cannot legally reuse your code.

Other useful files

- **CONTRIBUTING.md** - how to contribute
- **CODE_OF_CONDUCT.md** - community behaviour rules
- **SECURITY.md** - how to report vulnerabilities

- **.github/** folder - issue templates, PR templates, workflows

10. Major GitHub Features

Feature	What it does
GitHub Actions	CI/CD automation. Workflows are YAML files in .github/workflows/ that run on events like push or pull request (tests, builds, deployments).
GitHub Pages	Free static website hosting straight from a repo (portfolios, docs).
Codespaces	Cloud development environment (VS Code in the browser) with free monthly usage allowance.
GitHub Copilot	AI coding assistant for code suggestions, chat and agents (free tier and paid plans available).
Releases	Package and publish versions with notes and downloadable files, tied to tags.
Packages	Host npm, Docker, Maven, NuGet packages next to your code.
Gists	Share small code snippets or notes.
Dependabot	Automatically alerts about vulnerable dependencies and opens update PRs.
Code scanning / Secret scanning	Finds security bugs and accidentally committed API keys.
GitHub CLI (gh)	Use GitHub from the terminal: gh repo clone, gh pr create, gh issue list.
GitHub Desktop	Graphical app for Git and GitHub, good for beginners.
Stars, Watch, Follow	Bookmark repos, get notifications, follow developers.

Sample GitHub Actions workflow

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm test
```

Publishing a site with GitHub Pages

- Push your HTML/CSS/JS to a repo
- Go to Settings > Pages
- Choose the branch (for example main) and folder, then Save
- Your site appears at https://USERNAME.github.io/REPO/

11. Collaboration, Roles and Code Review

Repository access levels

Role	Capabilities
Read	View and clone; open issues and discussions
Triage	Manage issues and PRs without write access
Write	Push to branches, merge PRs
Maintain	Manage the repo without sensitive admin actions
Admin	Full control including settings and deletion

Code review

- Reviewers comment on specific lines, then Approve, Request changes or Comment.
- **Branch protection rules** can require reviews, passing checks and up-to-date branches before merging.
- **CODEOWNERS** file automatically requests reviews from the right people.
- **Draft pull requests** signal work in progress.

Public vs Private repositories

Public repos are visible to everyone; private repos only to people you grant access. Free accounts can have unlimited private repos. Organizations let teams own repos and manage permissions centrally.

12. Undoing Mistakes

Situation	Command
Discard unstaged changes in a file	git restore file.txt
Unstage a file (keep changes)	git restore --staged file.txt
Fix the last commit message / add forgotten file	git commit --amend
Undo last commit, keep changes staged	git reset --soft HEAD~1
Undo last commit, keep changes unstaged	git reset HEAD~1
Undo last commit and DELETE changes (dangerous)	git reset --hard HEAD~1
Safely undo a pushed commit by adding a new one	git revert <commit-hash>
Find lost commits	git reflog

Rule of thumb: use `revert` for shared history and `reset` only on local, unpushed work.

13. Resolving Merge Conflicts

A conflict happens when two branches change the same lines. Git pauses the merge and marks the file like this:

```
<<<<<<< HEAD
your version of the line
=======
their version of the line
>>>>>>> feature-branch
```

- Open the file and decide what the final text should be (keep one side, or combine).

- Delete the markers <<<<<<<, ======= and >>>>>>>.
- Run `git add <file>` then `git commit` to finish the merge.
- To cancel: `git merge --abort`.
- VS Code and GitHub's web editor both offer a visual conflict resolver.

14. Best Practices

Commit messages

- Write in the imperative: 'Add login form', not 'Added login form'
- Keep the first line under about 50 characters; add detail after a blank line
- One logical change per commit
- Popular style: Conventional Commits, such as `feat: add search bar` or `fix: handle empty input`

Branch naming

- `feature/short-description`
- `bugfix/issue-123-crash`
- `docs/update-readme`

Security hygiene

- **Never commit passwords, API keys or .env files.** If you do, rotate the key immediately; deleting the file later does not remove it from history.
- Turn on 2FA and prefer SSH keys or tokens with minimal scopes.
- Enable Dependabot and secret scanning on your repos.

General habits

- Commit early and often; pull before you start working
- Do not work directly on main; use branches and PRs
- Keep your README and docs current
- Pin a few polished projects on your profile and write a profile README (repo named exactly like your username)

15. Common Problems and Fixes

Problem	Fix
'Permission denied (publickey)'	SSH key not added to GitHub or agent. Re-check key setup or switch to HTTPS.
'Authentication failed' on HTTPS	Use a Personal Access Token or Git Credential Manager instead of your password.
'Updates were rejected' on push	Remote has new commits. Run git pull (or git pull --rebase), resolve conflicts, push again.
'fatal: not a git repository'	You are outside a repo folder, or need git init.

Problem	Fix
Committed to the wrong branch	git switch correct-branch, git cherry-pick <hash>, then reset the wrong branch.
Accidentally committed a large/secret file	Remove it, rotate secrets, and rewrite history with git filter-repo or BFG.
Detached HEAD	You checked out a commit. Run git switch -c new-branch to keep your work.
Wrong remote URL	git remote set-url origin <correct-url>

16. Quick Cheat Sheet

```
SETUP        git config --global user.name "Name"
             git config --global user.email "mail"
START        git init | git clone <url>
SNAPSHOT     git status | git add . | git commit -m "msg"
BRANCH       git branch | git switch -c name | git switch name
MERGE        git merge name | git merge --abort
REMOTE       git remote -v | git remote add origin <url>
SYNC         git fetch | git pull | git push -u origin name
INSPECT      git log --oneline --graph | git diff | git show <hash>
UNDO         git restore f | git restore --staged f | git revert <hash>
SHELVE       git stash | git stash pop | git stash list
FORK SYNC    git remote add upstream <url>
             git fetch upstream && git merge upstream/main
```

17. Glossary

Term	Definition
Origin / Upstream	Origin = your remote (often your fork). Upstream = the original repo you forked from.
Fast-forward	A merge where the branch pointer simply moves ahead, no merge commit needed.
Cherry-pick	Copy a single commit from one branch to another.
Rebase	Re-apply commits on top of another base to create linear history.
Squash	Combine multiple commits into one.
Hash (SHA)	Unique 40-character ID of a commit.
Tracked / Untracked	Whether Git is watching a file.
CI/CD	Continuous Integration / Continuous Delivery: automatic testing and deployment.
Markdown	Lightweight text format used in README, issues and PRs.
Maintainer	A person who manages and merges contributions to a project.
Hacktoberfest	Annual October event encouraging open-source contributions via pull requests.

Tip: practice is the fastest way to learn. Create a test repo, make branches, break things on purpose, and fix them.